Smart Electronics Lab v1.1

Guided breadboard learning system, communications console, and SDR workstation roadmap

Prepared for Bob Litt
Revised June 2026 — Design Review Pass

v1.1 · supersedes v1.0

0. What changed in v1.1

This revision keeps every decision from v1.0 (the feedback-loop concept, the three-stage roadmap, the BOM, and the lesson sequence) and closes the gaps a design review surfaced. Nothing below removes scope — it makes several "obviously fine" assumptions explicit so they don't become rework later.

1. **Stage 2 split into 2a/2b.** "Communications console" quietly mixed pedagogy (still teaching) with appliance features (just useful). Section 2 now names that split so the feedback-loop differentiator doesn't silently disappear partway through the roadmap.
2. **Graduation criteria added to the roadmap.** Figure 2 said "graduate" and "add compute" without saying what that means. Each stage transition now has a concrete, testable exit condition.
3. **New Section 3, "Open risks to resolve before Stage 2."** Audio-path feasibility on the Elecrow, the Stage1→Stage3 control interface, and the warn-vs-shutoff fault behavior were implied but undecided. They're now explicit open items with a recommended default.
4. **BOM now carries durable specs, not just links.** Each component row has an Interface/Spec column that survives an Amazon listing going stale. ASIN/links are collected separately in Section 15 and labeled as sourcing references, not the spec of record.
5. **Lesson content has a concrete schema.** Section 7 adds an actual YAML lesson record. Section 12 already named curriculum as the real commercial asset — it didn't have a format. Now it does.
6. **V1 fault behavior is defined, not just disclaimed.** Section 8 specifies exactly what the lesson engine does on over-current at V1, instead of only noting that auto-shutoff doesn't exist yet.
7. **Two lessons added.** L−1 (power-off-before-rewiring habit) before the first live circuit, and L8.5 (induced fault drill) so a learner sees a real fault warning under supervision before they see one by accident.

1. Executive decision

- The project is a guided electronics learning product, not a generic electronics kit.
- The Elecrow rotary display is the front-panel tutor. It displays steps, accepts knob/touch input, and becomes the control head for the communications console and later the SDR workstation.
- The breadboard stays standard. Smart behavior comes from power measurement and software, not from a proprietary smart breadboard.
- The first smart version uses off-the-shelf parts: a USB-C breadboard power supply, an inline USB-C power meter for visible measurement, and an I2C current/voltage sensor the Elecrow can read.

Short version: Buy the BusBoard BB830, USB-C power module, USB-C power meter, Adafruit INA219 current sensor, Qwiic/STEMMA cable kit, jumper wires, and a small set of components. Do not buy a giant Arduino kit.

2. Three-stage architecture REVISED

Stage 1
Smart Electronics Lab
Ohm's law, LEDs, buttons, pots, current measurement.
Elecrow displays the lesson.

→ graduate when L0–L9 run unattended (see exit criteria below)

Stage 2a / 2b
Console Build / Communications Console
2a: Wi-Fi & protocol lessons.
2b: internet radio, favorites, weather, ham reference.
Elecrow is the front panel.

→ add compute when an interface contract exists (Section 3.2)

Stage 3
SDR Workstation
15" monitor, Raspberry Pi / mini PC, RTL-SDR, waterfall, maps.
Elecrow becomes the control head.

Figure 2. Three-stage roadmap with graduation criteria (v1.1 adds the criteria; the stages are unchanged from v1.0).

Design rule (unchanged): every early purchase remains useful later. The breadboard teaches and prototypes. It does not become the final enclosure.

Stage 1: Smart Electronics Lab

Learn Ohm's law, LEDs, resistors, buttons, potentiometers, basic transistor switching, and simple audio. The display presents instructions and the smart power path measures whether the circuit behaves as expected.

Exit criteria: L0–L9 (Section 10) run end-to-end with the INA219 in the loop, and at least one beginner other than the builder completes the sequence without external help beyond the on-screen tutor.

Stage 2a: Guided Console Build SPLIT FROM STAGE 2

Same front panel and breadboard. New lessons teach Wi-Fi association, fetching a JSON API (e.g., weather), and basic protocol concepts — still using the instruction-plus-measurement loop, just measuring network/software state instead of only voltage and current.

Stage 2b: Communications Console SPLIT FROM STAGE 2

Internet radio, favorites, weather screens, local reference screens, basic ham information, and eventually an audio path. This is an appliance feature set built *on* the Stage 1/2a hardware and skills — it is not itself a measurement-feedback lesson. Treat it as a separate deliverable from 2a so "is this still teaching, or is it now a finished gadget?" has an explicit answer (it's the latter, and that's fine, as long as it's not budgeted as if it were lesson content).

Exit criteria: a still photo / short demo of the console playing an internet radio station from a saved favorite, on battery-independent USB-C power, with the same Elecrow unit used in Stage 1.

Stage 3: SDR Workstation

Add a Raspberry Pi or mini PC, the existing 15" monitor, and an RTL-SDR receiver. The heavy compute handles FFT, waterfall display, maps, decoding, and larger UIs. The Elecrow remains the control head.

Exit criteria: the Elecrow's knob and touch input visibly drive a parameter (e.g., tuned frequency or zoom) in an SDR application running on the separate compute, over whichever interface Section 3.2 settles on.

3. Open risks to resolve before Stage 2 NEW

None of these block ordering the Stage 1 parts list. They block committing Stage 2 scope, so they're worth a short spike now rather than a redesign later.

3.1 Audio path feasibility

Stage 2b assumes the Elecrow can stream and play internet radio. The confirmed spec (Section 15) lists UART, I2C, FPC expansion, 16M flash, and 8M PSRAM — it does not confirm an I2S output or amplifier. Decoding a compressed audio stream over Wi-Fi while also driving the touch UI is a real CPU/memory budget question on an ESP32-S3, not a given.

> **Recommendation:** before Stage 2b planning solidifies, run a 30–60 minute spike: confirm whether the Elecrow exposes I2S pins on its FPC/GPIO connector, and if not, budget for an add-on I2S DAC/amp module (e.g., MAX98357A class) in the Stage 2 BOM.

3.2 Stage 1 → Stage 3 control interface contract

"The Elecrow remains the control head" is a conclusion, not yet a design. If Stage 1/2 firmware hard-codes how knob/touch events are handled internally, that code won't necessarily transfer to "send this event to a Pi running GNU Radio."

Recommendation: adopt USB-serial (a simple line-based or JSON-per-line protocol over the existing USB-C data link) as the V1 contract for Stage 3, since it requires no new hardware and is easy to bring up. Define the message schema (e.g., `{"event":"knob","delta":-1}`) when Stage 2 firmware is written, even though it isn't used until Stage 3, so the event-handling code in Stage 1/2 already emits in that shape internally.

3.3 Warn-vs-shutoff fault behavior at V1

Section 8 (V1, software-observed power) can detect an over-current fault via the INA219 but cannot cut power. "Warn" needs an exact, specified behavior — see Section 8 below, which now defines it directly instead of leaving it implicit.

4. Stage 1 hardware design

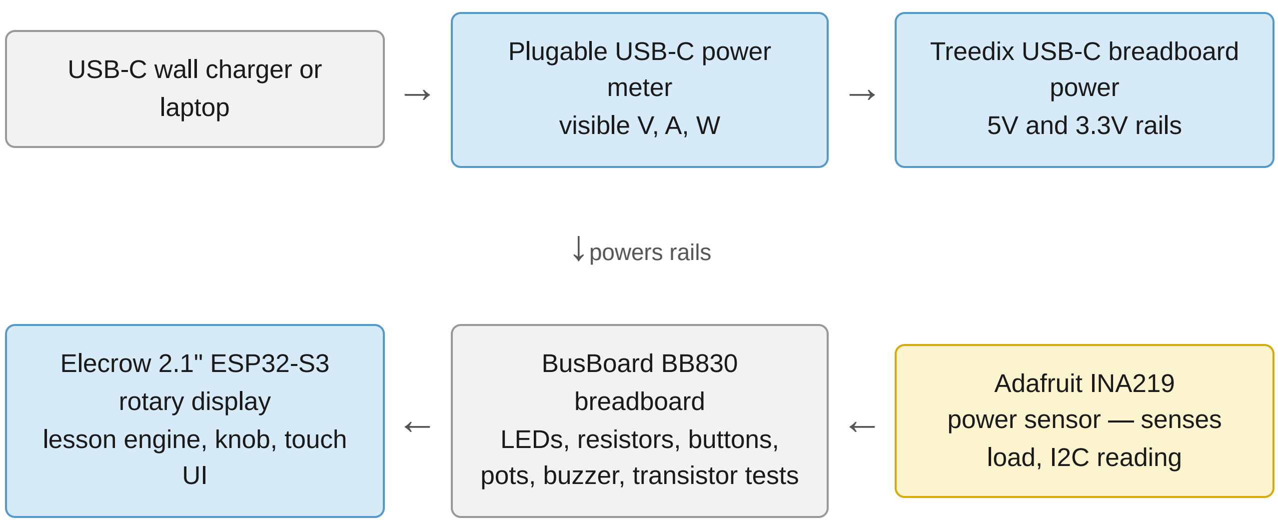


Figure 3. Stage 1 hardware architecture (unchanged from v1.0).

Version 0 is human-observed with the USB-C meter. Version 1 is software-observed with the INA219. Version 2 becomes a custom Smart Power Module.

Mounting answer

The Elecrow display does not need to sit in the breadboard. It sits next to the BB830 on the desk or mat, then connects to the breadboard through its GPIO/I2C connector or pigtail. The breadboard is for temporary circuits. The display is the tutor and controller.

Smart board answer

Do not start by designing a proprietary smart breadboard. Start by making the power path smart. If the Elecrow can read voltage and current from an INA219 sensor over I2C, the lesson engine can compare expected and measured behavior.

5. Bill of materials REVISED

v1.0 mixed durable specs with Amazon links in the same cell, which means the spec disappears if the listing changes. The table below keeps an **Interface / Spec** column that doesn't depend on any retailer; ASINs and links are collected in Section 15 as sourcing references only.

#	Category	Item	Interface / Spec	Qty / Timing	Rationale
1	Already bought	ELECROW ESP32 Rotary Display, 2.1" 480x480 IPS round, knob + touch	ESP32-S3, Wi-Fi/BLE, UART, I2C, FPC expansion, 16M flash, 8M PSRAM, 5V/1A input	1 / done	Front-panel tutor and future control head. Keep as main UI.
2	Core lab	BusBoard BB830 solderless breadboard	830 tie points, 4 power rails, 36V/2A rating, 21–26 AWG	1 / Buy now	Premium standard breadboard, correct size for lessons.
3	Power	Treedix USB-C breadboard power module	USB-C in, 3.3V/5V regulated rail out	1 pack / Buy now	First version of the lab power source.
4	Visible measurement	Plugable USB-C Power Meter Tester	Inline pass-through, OLED V/A/W readout	1 / Buy now	Makes Ohm's law visible immediately (V0).
5	Smart measurement	Adafruit INA219 current sensor, STEMMA QT	High-side DC, 26V target, ±3.2A, I2C, address-selectable	1 / Buy now	First real smart feedback module (V1).
6	Smart wiring	Qwiic/STEMMA QT cable kit with breadboard jumpers	JST-SH 4-pin to breadboard pins	1 / Buy now	Connects STEMMA/Qwiic sensors to the breadboard.

7	Wiring	Silicone flexible Dupont jumper wires	M-M, M-F, F-F	1 kit / Buy now	Far less stiff/annoying than standard kit wires.
8	Resistors	Aniann 1% metal film resistor assortment	1280 pcs, broad E-series coverage	1 / Buy now	Needed for every lesson.
9	LEDs	BOJACK 5mm LED assortment	5 colors	1 / Buy now	Makes current, polarity, and resistance visible.
10	Buttons	BOJACK tactile pushbutton assortment	10 values	1 / Buy now	Input lessons, later physical controls.
11	Potentiometers	MCIGICM 10K breadboard trim pots with knobs	10 pcs, 10KΩ	1 / Buy now	Voltage dividers, analog input, dimming.
12	Capacitors	EMGTMS ceramic capacitor kit	24 values, 480 pcs, 10pF–10μF	1 / Buy soon	RC timing, signal smoothing. Not needed for L1.
13	Electrolytic capacitors	Hilitchi electrolytic capacitor assortment	300 pcs	1 / Buy soon	Power smoothing, later audio work.
14	Transistors	PLUSIVO BJT assortment	NPN and PNP	1 / Buy soon	Switching larger loads from a GPIO pin.
15	Sound	Passive/active buzzer module pack	Arduino/ESP32 compatible	1 / Buy soon	Digital output, PWM/tone, audio cues.
16	Storage	Umirokin clear organizer boxes	2 pack, adjustable dividers	1 / Buy now	Keeps parts from becoming a loose bag.
17	Work surface	Anti-static silicone electronics mat	~16" x 12"	1 / Optional	Contains the bench. Not required.

6. What to buy first

- BusBoard BB830
- Treedix USB-C breadboard power module
- Plugable USB-C power meter

- Adafruit INA219 current sensor
- Qwiic/STEMMA cable kit
- Silicone jumper wires
- Resistor kit
- LED kit
- Button kit
- 10K potentiometers
- Organizer box

Defer: capacitor kits, transistor kit, and buzzer can be bought now for a complete Stage 1 box, but are not necessary for the first two or three lessons.

7. Software loading and lesson content REVISED

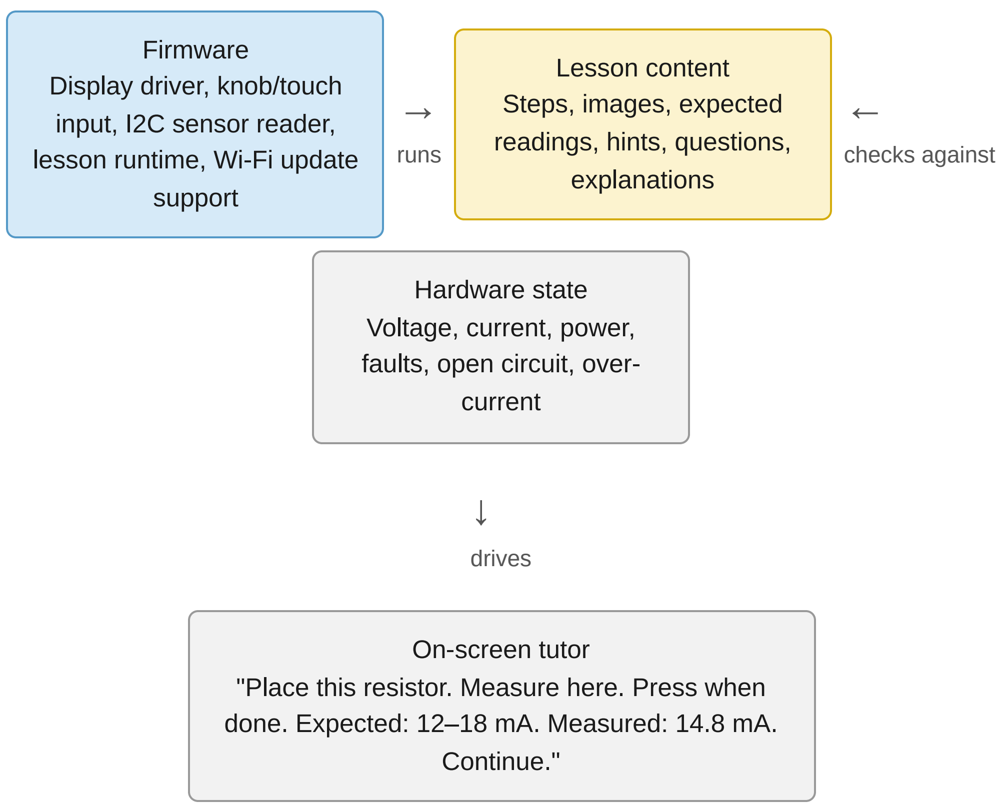


Figure 4. Software architecture for the tutor and lesson packs (unchanged structure; content format below is new).

Lesson schema NEW

Section 12 (Product concept) calls curriculum the real commercial asset and "course packs" the scaling mechanism. That only works if a lesson is a structured record, not a one-off firmware routine.

A minimal schema for the lesson engine in Figure 4:

```
id: L1_led_with_resistor
title: "LED with resistor"
steps:
  - prompt: "Place a 330 ohm resistor from row A10 to A15."
    wait_for: knob_press
  - prompt: "Place the LED long leg at A15, short leg to the ground rail."
    wait_for: knob_press
  - prompt: "Turn power on."
    measure: current_mA
    expected_range: [8, 15]
    on_pass: { advance: true, message: "Correct. Press to continue." }
    on_fail:
      open_circuit: "No current detected. Check the ground rail connection."
      over_range: "Current is higher than expected. Check resistor value and LED polarity
hints:
  - "LED current is set mostly by the resistor, not the LED."
next: L2_change_the_resistor
```

This is deliberately small. The fields map directly to Figure 4's "Lesson content" box (steps, images, expected readings, hints) and "Hardware state" box (the `measure` / `expected_range` / fault keys). New course packs (Section 12) are new files in this format, not new firmware.

Developer mode

Connect the Elecrow to a Mac or PC with a USB-C data cable. Use PlatformIO first if possible, or Arduino IDE if following Elecrow examples. Upload firmware over USB-C. This is how the first prototypes should be built.

User mode

Later, the device should update over Wi-Fi. A user would choose a lesson pack from a menu and the device would download content. This is not Stage 1 work.

MicroSD answer

Do not buy a microSD card for this board yet. The Elecrow listing specifies 16M flash, UART, I2C, and FPC expansion, but does not advertise a microSD slot. For v1, keep lesson content (in the schema above) in firmware or internal flash. Add external SD only if hardware inspection confirms a usable slot, or add an SPI SD module later.

Lesson engine

The correct architecture is a small lesson engine, not custom firmware per lesson. The engine knows how to display a step, draw a wiring diagram, wait for a knob press, ask a question, read the current sensor, compare expected values against the schema above, and advance.

8. Smart Power Module path REVISED

V0: Visible power

Use the Plugable USB-C meter. The learner reads voltage, current, and watts manually. Sufficient for Ohm's law lessons.

V1: Software-observed power

Wire the INA219 in series with the breadboard load, connected to the Elecrow via I2C. The lesson engine reads current and voltage directly.

V1 fault behavior (defined, not implicit): on over-current or out-of-range read, the lesson engine (1) shows a full-screen red fault banner, (2) displays the explicit instruction "Disconnect power now," and (3) logs the fault event with timestamp and reading. There is no automatic cutoff at V1 — the learner is the disconnect mechanism. This is acceptable for a supervised lab but should be stated to anyone using the device unsupervised, and is the reason L8.5 (Section 10) exists: a learner should see this exact banner once, deliberately, before encountering it by accident.

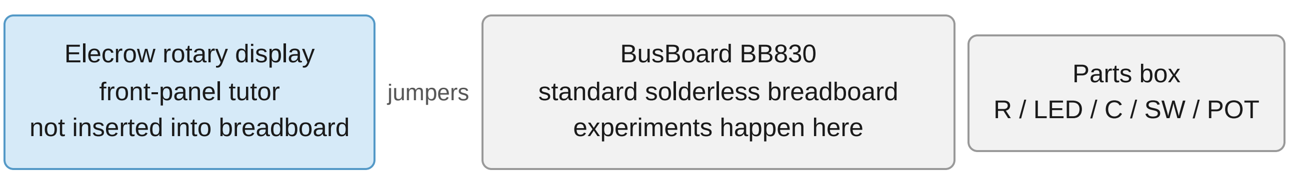
V2: Protected smart module

Design a custom board with USB-C input, current limiting, measurement, fault reporting, and a safe disconnect circuit (e.g., a load switch IC gated by the fault condition). This becomes the commercial product core, and is the first version where "warn" can become "warn and cut."

V3: Row-aware smart breadboard

Only after the power module works, explore sensing rows or continuity. This is much harder and should not be the first product.

9. Recommended desk layout for v1



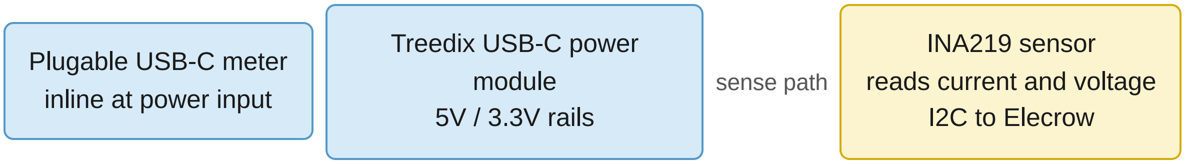


Figure 5. Recommended desk layout (unchanged from v1.0).

10. First guided lesson sequence REVISED

L−1: Power-off habit NEW — before any live circuit: the screen asks the learner to disconnect power, then walks through placing/removing a single jumper, then reconnect. No measurement yet. This converts Section 11's safety assumption ("disconnect power before rewiring") from a written rule into a practiced habit before it matters.

L0: Hello Lab: Load firmware over USB-C. The display shows a welcome screen, knob movement, and press-to-advance.

L1: LED with resistor: Build a simple LED circuit. The screen teaches polarity, current limiting, and expected current.

L2: Change the resistor: Swap 1k, 470Ω, and 220Ω resistors. The screen asks the learner to predict brightness and current before measuring.

L3: Series resistance: Place two resistors in series and measure lower current.

L4: Parallel branches: Build two LED branches and observe total current increase.

L5: Button input: Wire a pushbutton and pull-down resistor. The display changes when the button is pressed.

L6: Potentiometer: Use a 10K pot as a voltage divider. Turn the pot and watch analog voltage change.

L7: Software-controlled LED: Use the Elecrow to drive an LED through a resistor. Then map knob position to brightness.

L8: Smart measurement: Insert the INA219 into the load path. The display compares expected and measured current.

L8.5: Induced fault drill NEW — the lesson deliberately has the learner swap in a resistor value low enough to trigger the V1 over-current fault banner (Section 8) under supervision, then walk through the "disconnect power now" response. The point is to make the fault screen familiar and procedural, not alarming, the first time it appears for real.

L9: Transistor switch: Use a transistor to drive a buzzer or higher-current LED path from a GPIO pin.

11. Example on-screen lesson flow

1. Screen: Find a 330Ω resistor. Press the knob when found.
2. Screen: Place one end at row A10 and the other at row A15.
3. Screen: Place LED long leg at A15 and short leg to the ground rail.
4. Screen: Connect 5V rail through the resistor path. Keep power off until the next step.
5. Screen: Turn power on. Expected current is about 8 to 15 mA depending on LED color and supply voltage.
6. Screen, V0: Read the Plugable meter and enter or select the closest current.
7. Screen, V1: Sensor reads 11.4 mA. Correct. Press to continue.
8. Screen: Replace the 330Ω resistor with 1k. Predict whether the LED gets brighter or dimmer.

12. Product concept REVISED

What is distinctive

Most electronics kits give instructions. This lab gives feedback. The device does not merely say what to build. It observes voltage and current and tells the learner what likely happened.

Why this can be a product

The first product does not require a proprietary breadboard. The first sellable unit could be a smart USB-C power and measurement module, plus an ESP32 display tutor, plus a standard BB830.

Course packs

The same hardware could support course packs: Electronics Fundamentals, Internet Radio, Amateur Radio, SDR Basics, Sensors, Home Automation, and Robotics. With the lesson schema in Section 7, each course pack is a set of schema files plus assets, not a firmware fork — this is what makes "course pack" a real product line rather than a slogan.

Commercial risk

The difficult parts are not the first hardware. The hard work is curriculum design, safe fault handling, reliable connectoring, and making lessons resilient when beginners miswire circuits. The lesson schema and the V1 fault behavior (Sections 7–8) are the first concrete artifacts against that risk, not just statements of it.

13. Safety scope

- Stage 1 is low-voltage DC only: 3.3V and 5V breadboard experiments.
- No AC mains. No wall outlet experiments. No exposed lithium battery charging experiments.

- Assume beginner mistakes. Use current limiting, start with higher resistor values, and disconnect power before rewiring — now practiced directly in L−1.
- The smart module should detect over-current and shut off or warn before damage occurs. At V1, the off-the-shelf module can only warn (Section 8); a learner has practiced responding to that warning by L8.5 before it can happen unsupervised.

14. Next build action after parts arrive REVISED

1. Photograph the Elecrow connector set that arrives in the box. The exact pigtail determines the cleanest breadboard connection.
2. Load a basic Elecrow example over USB-C. Confirm the knob, press, touch, and display work.
3. **Check for I2S/audio pins on the Elecrow's FPC/GPIO breakout** NEW — resolves Section 3.1 before Stage 2b BOM is finalized.
4. Assemble the BB830 rails with the Treedix power module. Verify 5V and 3.3V with the multimeter before connecting the Elecrow.
5. Build the first LED/resistor lesson manually using the USB-C meter for visible feedback.
6. Add the INA219 and make the Elecrow display current and voltage. That is the first true Smart Lab milestone.
7. **Draft the Stage 3 control-interface message schema** NEW — even a one-page note, so Stage 2 firmware's event handling already emits knob/touch events in that shape (Section 3.2).

15. Sourcing references RENAMED from "Source notes"

These are retailer listings, not the spec of record (see the Interface/Spec column in Section 5). Prices, stock, and listing details change; if a link goes stale, search by ASIN or product name.

- **Elecrow 2.1" rotary display** (ASIN B0G3TZGRC4): rotary knob, capacitive touch, ESP32-S3, Wi-Fi/BLE, UART, I2C, FPC expansion, 16M flash, 8M PSRAM, 5V/1A input.
- **BusBoard BB830** (ASIN B0040Z4QN8): 830 tie points, 4 power rails, 6.5 x 2.2 x 0.3 in, 36V/2A rating, 21–26 AWG insertion wire size.
- **Treedix USB-C breadboard power module** (ASIN B0CN6HYCJD): USB-C charging/power, 3.3V and 5V outputs.
- **Plugable USB-C power meter** (ASIN B0B5W5NKKN): voltage and amperage measurement, power monitoring, bidirectional connection, data passthrough.
- **Adafruit INA219** (ASIN B09CBSLXN7): high-side current measurement, 26V target voltage, ±3.2A range, I2C address options.
- **Adafruit INA260** (official page): higher-capacity future option, integrated shunt, I2C, 3V/5V logic compatible, up to 36V/15A. Candidate for the V2 custom Smart Power Module.